SIMATS ENGINEERING

ASSESSMENT TOOL 3

NS AVESH HUSSAIN
192525241

COURSE NAME: **Database Management Systems**
COURSE CODE: **CSA05**

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 30%

Dr

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	**5**
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	**5**
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	**5**
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	**5**
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	**5**
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	**5**

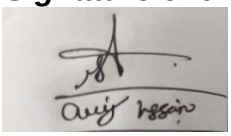
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	**5**

8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	**5**
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINs and using appropriate indexes.	BL6 – Create	**5**
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	**5**

Code of Conduct:

I N S A V E S H H U S S A I N (1 9 2 5 2 5 2 4 1) ____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues

Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and	Shows reasonable analysis with	Limited analysis; weak reasoning	No analytical reasoning demonstrated

		clear justification of solutions	some justification		
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Criteria	**Weightage (%)**	**Level 4 (Exemplary) (85-100%)**	**Level 3 (Proficient) (70-84%)**	**Level 2 (Developing) (50-69%)**	**Level 1 (Beginning) (0-49%)**	**Marks Awarded (Out of)**
1. Problem Understanding	20%	☐	☐	☐	☐	_____
2. Method Selection	20%	☐	☐	☐	☐	_____
3. Solution Process	25%	☐	☐	☐	☐	_____
4. Accuracy of Calculation	20%	☐	☐	☐	☐	_____
5. Interpretation of Results	15%	☐	☐	☐	☐	_____
Total Marks	**100%**					_____ / 50

1. Incorrect JOIN Syntax Producing Duplicate Results

Introduction

JOIN is one of the most important SQL operations used to combine data from two or more tables based on a common column. Incorrect JOIN conditions may produce duplicate records, missing records, or incorrect results. Proper JOIN conditions improve query accuracy and database performance.

Answer

When a JOIN statement does not specify the correct relationship between tables, duplicate rows are generated. This usually happens because the matching condition is missing, incorrect, or multiple matching records exist in the joined tables. The first step in debugging is to identify the tables involved and verify the relationship between their primary and foreign keys.

After identifying the error, the JOIN condition should be corrected so that only related records are matched. Using appropriate JOIN types such as INNER JOIN, LEFT JOIN, RIGHT JOIN, or FULL JOIN based on the requirement also helps avoid unnecessary records. Duplicate records can be reduced by using DISTINCT when needed and by ensuring the database design follows normalization principles.

Optimizing JOIN queries improves execution speed, reduces memory usage, and produces accurate results. Database indexes on commonly joined columns further improve performance.

Key Points

- JOIN combines data from multiple tables.
- Incorrect JOIN conditions produce duplicate rows.
- Verify primary and foreign key relationships.
- Use the correct JOIN type.
- Avoid Cartesian products.
- Use DISTINCT only when necessary.
- Create indexes on JOIN columns.
- Normalize database tables.
- Test query output after debugging.
- Improve query performance.

Advantages

- Accurate query results.
- Eliminates duplicate records.
- Improves execution speed.
- Better database performance.
- Efficient data retrieval.

Applications

- Banking systems.
- Hospital management.
- Library management.
- E-commerce websites.
- Student management systems.

2Q)

A sub-query is a query written inside another SQL query. A **correlated sub-query** is a sub-query that depends on values from the outer query. If the correlation is missing, the sub-query returns incorrect or unexpected results because it is executed independently of the outer query. Debugging such queries helps improve both correctness and performance.

Answer

A missing correlation occurs when the inner query is not linked to the outer query using a common column. As a result, the sub-query may return all rows or incorrect rows instead of only the matching records. During debugging, the first step is to identify whether the sub-query references the required columns from the outer query.

After identifying the issue, the correlation between the outer query and the sub-query should be established using the appropriate common attribute. This ensures that the sub-query executes separately for each row of the outer query and returns only the relevant data.
To optimize performance, correlated sub-queries can be replaced with JOIN operations whenever possible. Proper indexing on related columns also improves execution speed. After debugging, the query should be tested with different datasets to verify that it produces accurate and consistent results.

Key Points

- A sub-query is a query inside another query.
- A correlated sub-query depends on the outer query.
- Missing correlation produces incorrect results.
- Identify the common column between queries.
- Link the inner and outer queries correctly.
- Validate the query using sample data.
- Replace sub-queries with JOINs when suitable.
- Use indexes for better performance.
- Test the query after debugging.
- Ensure accurate and efficient query execution.

Advantages

- Produces accurate query results.
- Improves query performance.
- Reduces unnecessary data retrieval.
- Simplifies database maintenance.
- Enhances overall database efficiency.

Applications

- Banking systems.
- Hospital management systems.
- E-commerce applications.
- Library management systems.
- Student information systems.

3Q)
A SQL view may produce errors due to incorrect table names, invalid column names, missing relationships between tables, or unsupported SQL statements. Another common issue is creating a view without proper aliases or selecting columns that do not exist in the base tables. These mistakes prevent the database from retrieving accurate information.
To debug the view, the first step is to verify the table structure and ensure that all column names are correct. Next, confirm that the relationships between the tables are properly defined. The view should include only the required columns instead of selecting unnecessary data, which improves query performance and reduces memory usage.
An optimized view should have a clear structure, meaningful column names, and only the necessary data. It should avoid complex nested queries whenever possible and make use of indexed columns to improve execution speed. After correcting the definition, the view should be tested to ensure that it returns the expected results without errors.
Views also provide security by allowing users to access only specific columns instead of the complete table. They simplify complex database operations by hiding unnecessary implementation details from end users.

Key Points

- A view is a virtual table.
- It is created from one or more tables.
- Incorrect view definitions cause query errors.

- Verify table and column names.
- Check relationships between tables.
- Select only required columns.
- Avoid unnecessary complex queries.
- Improve performance using indexed columns.
- Test the view after modification.
- Ensure accurate and secure data retrieval.

Advantages

- Simplifies complex queries.
- Improves data security.
- Provides faster data access.
- Reduces query complexity.
- Enhances database performance.

Applications

- Banking Management Systems.
- Hospital Management Systems.
- Library Management Systems.
- Student Information Systems.
- E-Commerce Applications.

4Q)

A Natural Join automatically joins two relations using their common attribute names. If the common attributes are missing, incorrectly named, or contain inconsistent values, the join operation may return incorrect results. Sometimes, duplicate attribute names or mismatched data types also cause errors in the output.
To debug the expression, the first step is to identify the common attributes used for the Natural Join. Next, verify that both relations contain matching attribute names and compatible data types. The relations should also have valid and consistent values in the common columns.
The expression should be modified so that the Natural Join is performed only on the correct matching attributes. If required, attribute names should be renamed before performing the join to avoid ambiguity. After correcting the expression, the result should be tested to ensure that only the required records are returned without duplication.
Optimizing the Natural Join improves query execution, reduces unnecessary data processing, and ensures that meaningful information is retrieved from multiple relations.

Key Points

- Relational Algebra is a procedural query language.
- Natural Join combines two relations using common attributes.
- Incorrect attribute names produce wrong results.
- Verify common columns before joining.
- Ensure compatible data types.
- Remove duplicate attributes.
- Rename attributes when necessary.
- Test the expression after debugging.
- Improves query accuracy.
- Enhances database performance.

Advantages

- Produces accurate query results.
- Eliminates duplicate data.
- Improves query performance.
- Simplifies data retrieval.
- Supports efficient database operations.

Applications

- Student Management Systems.
- Banking Management Systems.
- Hospital Management Systems.
- Library Management Systems.
- Inventory Management System

5Q)

A Tuple Relational Calculus expression may produce incorrect results when tuple variables are not properly declared, selection conditions are missing, or logical operators are used incorrectly. Another common problem is referencing attributes that do not belong to the selected relation or failing to specify relationships between multiple relations.
To debug the expression, the tuple variables should first be verified to ensure they represent the correct relations. Next, all conditions should be checked to confirm that the required attributes and comparison operators are used correctly. Logical operators such as **AND**, **OR**, and **NOT** should be applied appropriately so that the query returns only the intended tuples.
The corrected TRC expression should clearly specify the required conditions and relationships among the relations. After modification, the expression should be tested with sample data to verify that it returns accurate and complete results. Optimizing the expression reduces unnecessary processing and improves query performance.

Key Points

- TRC is a non-procedural query language.
- It specifies **what** data is required.
- Tuple variables represent rows in a relation.
- Incorrect conditions produce wrong results.
- Verify tuple variables before execution.
- Check logical operators carefully.
- Ensure valid attribute references.
- Maintain correct relationships between relations.
- Test the expression after debugging.
- Optimize the query for better performance.

Advantages

- Easy to express complex queries.
- Improves query accuracy.
- Supports efficient data retrieval.
- Reduces logical errors.
- Enhances database performance.

Applications

- Student Management Systems.
- Banking Systems.
- Hospital Management Systems.
- Library Management Systems.
- E-Commerce Applications.

6Q)

A query execution plan shows the sequence of operations performed by the database while executing a query. If the plan indicates full table scans, unnecessary joins, or inefficient sorting operations, the query may take more time and consume additional system resources.
To optimize the query, the first step is to examine whether indexes are being used. Creating indexes on frequently searched or joined columns reduces data access time. Unnecessary columns should be avoided by selecting only the required attributes instead of retrieving all columns. Complex queries can be simplified by reducing nested subqueries and using appropriate JOIN operations. Filtering records using the WHERE clause before performing joins decreases the number of rows processed. Database statistics should also be updated regularly so that the query optimizer can choose the best execution plan.
Finally, the optimized query should be tested again using the execution plan to verify improvements in performance. Proper optimization reduces execution time, minimizes memory usage, and improves overall database efficiency.

Key Points

- Query Execution Plan shows how a query is executed.
- Helps identify performance bottlenecks.
- Detects full table scans.
- Uses indexes for faster retrieval.
- Select only required columns.
- Reduce unnecessary joins.
- Optimize subqueries.
- Apply filtering using the WHERE clause.
- Update database statistics.
- Test performance after optimization.

Advantages

- Improves query performance.
- Reduces execution time.
- Minimizes memory usage.
- Enhances database efficiency.
- Supports faster data retrieval.

Applications

- Banking systems.
- Hospital management systems.
- E-commerce websites.
- Library management systems.
- Student information systems.

7Q)
A materialized view may fail to refresh because of incorrect refresh settings, missing privileges, changes in the base tables, or database errors. When the underlying data changes but the materialized view is not refreshed, users receive outdated results. This affects the accuracy and reliability of reports.
To debug the problem, the refresh method should first be verified. The database administrator should check whether the materialized view is configured for **complete refresh**, **fast refresh**, or **manual refresh**. If the required logs or dependencies are missing, the refresh operation may fail.
The next step is to verify the base tables and ensure that they are updated correctly. Database permissions should also be checked to confirm that the user has sufficient privileges to refresh the materialized view. Any errors recorded in the database logs should be identified and corrected.
For better performance, refresh the materialized view at appropriate intervals based on application

requirements. Fast refresh can be used when only incremental changes need to be updated, while complete refresh is suitable when the entire dataset must be regenerated. After applying these changes, the materialized view should be tested to ensure it displays the latest data correctly.

Key Points

- Materialized views store query results physically.
- They improve query performance.
- Incorrect refresh leads to outdated data.
- Verify refresh method.
- Check base table updates.
- Ensure required permissions.
- Review database error logs.
- Use fast refresh when appropriate.
- Schedule regular refresh operations.
- Test the view after refreshing.

Advantages

- Improves query performance.
- Provides faster report generation.
- Reduces execution time.
- Stores precomputed results.
- Supports efficient data analysis.

8Q)

A deadlock usually occurs when multiple transactions access the same resources in different orders. For example, one transaction locks Resource A and waits for Resource B, while another transaction locks Resource B and waits for Resource A. Since both transactions are waiting for each other, neither can proceed, resulting in a deadlock.
To identify a deadlock, the database management system monitors transaction activities and detects circular waiting conditions. Once a deadlock is detected, the DBMS resolves it by rolling back or terminating one of the transactions, allowing the other transaction to continue.
Several techniques can be used to prevent deadlocks. Transactions should access resources in the same order to avoid circular waiting. Transactions should also be kept short so that locks are held for a minimum amount of time. Lock timeouts can be configured to automatically release transactions that wait too long. Proper indexing and efficient query design reduce transaction execution time and minimize the chances of deadlocks.
Database administrators should continuously monitor transaction logs and optimize database operations to ensure smooth execution. These practices improve concurrency, maintain data consistency, and increase overall database performance.

Key Points

- Deadlock occurs when transactions wait for each other.
- It causes transactions to stop executing.
- Circular waiting is the main cause.
- DBMS detects deadlocks automatically.
- One transaction is rolled back to resolve the deadlock.
- Access resources in the same order.
- Keep transactions short.
- Use lock timeout mechanisms.
- Monitor transaction logs regularly.
- Optimize queries to reduce lock time.

Advantages of Deadlock Prevention

- Improves database performance.
- Increases transaction efficiency.
- Reduces waiting time.
- Ensures data consistency.
- Supports better concurrency.

Applications

- Banking systems.
- Online shopping applications.
- Airline reservation systems.
- Hospital management systems.
- Inventory management systems.

9Q)

Database redundancy occurs when the same data is stored in multiple places within the database. This increases storage requirements and makes it difficult to maintain consistent information. If one copy of the data is updated while another is not, inconsistencies arise, leading to incorrect results.

To improve the database design, normalization should be applied. The **First Normal Form (1NF)** removes repeating groups and ensures that each column contains only atomic values. The **Second Normal Form (2NF)** removes partial dependencies by ensuring that non-key attributes depend on the entire primary key. The **Third Normal Form (3NF)** eliminates transitive dependencies so that non-key attributes depend only on the primary key.

Normalization reduces data duplication, improves consistency, and simplifies database maintenance. It also minimizes update, insertion, and deletion anomalies. However, excessive normalization may increase the number of table joins, so database designers should maintain a balance between normalization and performance.

A well-normalized database improves data quality, simplifies maintenance, and ensures reliable information retrieval.

Key Points

- Normalization organizes database tables.
- Reduces data redundancy.
- Eliminates update anomalies.
- Prevents insertion anomalies.
- Prevents deletion anomalies.
- First Normal Form (1NF) removes repeating groups.
- Second Normal Form (2NF) removes partial dependency.
- Third Normal Form (3NF) removes transitive dependency.
- Improves data consistency.
- Enhances database performance.

Advantages

- Reduces duplicate data.
- Improves data integrity.
- Saves storage space.
- Simplifies database maintenance.
- Produces accurate query results.

Applications

- Banking Management Systems.
- Hospital Management Systems.
- Student Information Systems.
- Library Management Systems.
- E-Commerce Databases.

10Q)

Large database tables often require more time to retrieve records because the database has to scan many rows before finding the required data. This problem can occur due to missing indexes, inefficient SQL queries, unnecessary columns, or lack of proper optimization.
The first optimization technique is to create **indexes** on frequently searched or joined columns. Indexes help the database locate records quickly without scanning the entire table. Another technique is to retrieve only the required columns instead of all columns, which reduces data processing and improves query speed.
Efficient filtering using appropriate search conditions helps reduce the number of records processed. Frequently accessed data can also be stored in cache to improve response time. Partitioning very large tables into smaller logical sections further improves query performance. Regular database maintenance, such as updating statistics and removing unused data, ensures that the query optimizer chooses the most efficient execution plan.
Monitoring database performance and analyzing query execution plans help identify bottlenecks and optimize slow-running queries. These techniques improve response time, reduce system load, and provide better user experience.

Key Points

- Large tables increase query execution time.
- Create indexes on frequently searched columns.
- Retrieve only the required columns.
- Use efficient filtering conditions.
- Optimize SQL queries.
- Partition very large tables.
- Update database statistics regularly.
- Remove unnecessary or duplicate data.
- Analyze query execution plans.
- Monitor database performance continuously.

Advantages

- Faster data retrieval.
- Reduced query execution time.
- Better utilization of system resources.
- Improved database performance.
- Enhanced user experience.

Applications

- Banking Management Systems.
- Hospital Management Systems.
- E-Commerce Websites.
- Library Management Systems.
- Student Information Systems.